

Lexora

Le Backend expliqué pas à pas
Guide pour débuter en programmation backend

Node.js · Express · SQLite · JWT
Document pédagogique — 2026

1. Avant de commencer : c'est quoi un « backend » ?

Quand tu ouvres une application web, il y a en réalité **deux programmes** qui travaillent ensemble :

Partie	Rôle	Dans Lexora
Frontend (le « devant »)	Ce que l'utilisateur voit et clique : boutons, formulaires, tableaux. Tourne dans le navigateur.	React (dossier frontend/)
Backend (le « derrière »)	Le cerveau caché : il reçoit les demandes, lit/écrit dans la base, applique les règles et renvoie des données. Tourne sur un serveur.	Node.js + Express (dossier backend/)

Le frontend ne touche **jamais** directement à la base de données. Il demande poliment au backend : « donne-moi la liste des tâches », « crée cette facture ». Le backend répond. Cet échange s'appelle une **API**.

Image mentale : le frontend est un client au restaurant, le backend est la cuisine, et la base de données est le frigo. Le client ne fouille jamais dans le frigo lui-même : il passe commande au serveur (l'API), qui va en cuisine.

L'API REST en une phrase

Une **API REST**, c'est un ensemble d'adresses (URL) que le frontend peut appeler, en utilisant des **verbes HTTP** pour dire ce qu'il veut faire :

Verbe HTTP	Intention	Exemple
GET	Lire / récupérer	Lister toutes les tâches
POST	Créer	Ajouter une nouvelle tâche
PUT	Modifier	Changer le statut d'une tâche
DELETE	Supprimer	Effacer une tâche

Les données échangées sont au format **JSON** (du texte structuré, lisible par les humains et les machines) :

```
{ "titre": "Préparer la réunion", "statut": "todo", "assignee": "Marie" }
```

2. La pile technique de Lexora

Outil	À quoi ça sert
Node.js	Permet d'exécuter du JavaScript en dehors du navigateur, côté serveur.
Express	Boîte à outils pour Node qui simplifie la création d'API (routes, lecture des requêtes...).
SQLite (better-sqlite3)	Base de données stockée dans un simple fichier (lexora.db). Pas de serveur séparé à installer.
JWT (jsonwebtoken)	Système de « badge d'accès » pour savoir qui est connecté (authentification).
bcryptjs	Chiffre les mots de passe pour ne jamais les stocker en clair.
helmet / cors / rate-limit	Trois protections de sécurité (voir section 4).
multer	Gère l'upload de fichiers (coffre-fort documentaire).

Organisation des fichiers du backend

```
backend/
■■■ index.js          <- Point d'entrée : démarre le serveur
■■■ env.js            <- Charge les variables secrètes (.env)
■■■ models/
■   ■■■ db.js         <- Crée la base de données et les tables
■■■ middleware/
■   ■■■ auth.js       <- Vérifie le badge JWT
■■■ services/
■   ■■■ ollamaService.js <- Parle à l'IA (Ollama)
■■■ routes/          <- Un fichier = un sujet
    ■■■ auth.js       <- Connexion
    ■■■ taches.js     <- Tâches projet
    ■■■ todos.js      <- Notes rapides
    ■■■ factures.js   <- Factures
    ■■■ clients.js    <- Clients
    ■■■ employes.js   <- Employés (protégé)
    ■■■ planning.js   <- Plannings
    ■■■ evenements.js <- Calendrier
    ■■■ documents.js  <- Coffre-fort (fichiers)
    ■■■ automatisations.js <- Automatisations (protégé)
    ■■■ assistant.js  <- Assistant IA
```

Principe clé : « un fichier de route = un domaine métier ». Si tu veux modifier la gestion des factures, tu sais directement qu'il faut ouvrir [routes/factures.js](#). C'est une règle d'or pour garder un projet lisible.

3. Le point d'entrée : index.js

C'est le fichier qui se lance en premier (`npm start`). Il assemble toutes les pièces. Décortiquons-le.

3.1 — Création du serveur

```
import './env.js';           // 1. Charge les variables secrètes en premier
import express from 'express';
const app = express();       // 2. Crée l'application
const PORT = process.env.PORT || 3000;
```

- `import './env.js'` doit être la **première ligne** : il lit le fichier `.env` (clé JWT, etc.) AVANT que les autres fichiers en aient besoin.
- `express()` crée l'objet `app` qui représente tout notre serveur.
- `PORT` est le « numéro de porte » sur lequel on écoute (3000 par défaut).

3.2 — Les middlewares (lignes 39 à 73)

Un **middleware** est une fonction qui s'exécute *avant* d'atteindre ta route, sur chaque requête. Imagine une série de portiques de sécurité que chaque visiteur traverse.

```
app.use(helmet());           // Ajoute des en-têtes de sécurité HTTP
app.use(cors({ ... }));      // Autorise seulement certains sites
app.use('/api/', limiter);    // Bloque si trop de requêtes (anti-spam)
app.use(express.json());      // Traduit le JSON reçu en objet JavaScript
```

Ligne très importante : `app.use(express.json())`. Sans elle, `req.body` serait `undefined` : le backend ne saurait pas lire les données envoyées par le frontend dans un POST. C'est une source d'erreur classique chez les débutants.

3.3 — Le montage des routes (lignes 81 à 91)

```
app.use('/api/auth', authRouter);
app.use('/api/taches', tachesRouter);
app.use('/api/factures', facturesRouter);
// ... etc
```

Cette ligne dit : « toutes les URL qui commencent par `/api/taches` seront gérées par le fichier `routes/taches.js` ». C'est le branchement entre l'URL et le code.

3.4 — Le démarrage

```
app.listen(PORT, () => {
  console.log(`Lexora backend running on port ${PORT}`);
});
```

`app.listen` allume le serveur. À partir de là, il « écoute » et attend les requêtes. Le message s'affiche une seule fois, au démarrage.

4. La sécurité (à comprendre tôt !)

Protection	Contre quoi	Comment
helmet	Attaques via les en-têtes HTTP	Ajoute automatiquement ~14 en-têtes de sécurité.
CORS	Sites malveillants appelant l'API	Seules les adresses de <code>ALLOWED_ORIGINS</code> sont acceptées.
rate-limit	Brute-force, DDoS basique	Max 100 requêtes par IP toutes les 15 min.
bcrypt	Vol de mots de passe	Les mots de passe sont « hachés » avant stockage.
JWT	Accès non autorisé	Un jeton signé prouve l'identité à chaque requête protégée.

Comment marche l'authentification JWT ?

1. L'utilisateur envoie email + mot de passe -> `POST /api/auth/login`
2. Le backend vérifie le mot de passe avec `bcrypt`
3. Si OK, il fabrique un token (jeton signé) valable 24h
4. Le frontend stocke ce token (dans le navigateur)
5. À chaque requête protégée, le frontend renvoie le token dans l'en-tête : `Authorization: Bearer <token>`
6. Le middleware `verifyJWT` vérifie le token avant de laisser passer

Le fichier `middleware/auth.js`

```
export function verifyJWT(req, res, next) {
  const header = req.headers['authorization'];
  if (!header?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Token manquant' }); // pas de badge
  }
  const token = header.slice(7); // enlève le mot "Bearer "
  try {
    req.admin = jwt.verify(token, process.env.JWT_SECRET); // badge valide ?
    next(); // OK -> on passe à la route
  } catch {
    res.status(401).json({ error: 'Token invalide ou expiré' });
  }
}
```

- `next()` est le mot magique : il dit « tout va bien, continue vers la route ».
- Si on appelle `res.status(...).json(...)` à la place, la requête est **arrêtée net** : la route protégée n'est jamais exécutée.
- Le code **401** signifie « non authentifié ».

Ce middleware protège les routes `/api/employees` et `/api/automations` (voir `index.js` lignes 88-89).

5. La base de données : models/db.js

Ce fichier fait trois choses au démarrage :

- **Ouvre** (ou crée) le fichier `lexora.db`.
- **Crée toutes les tables** si elles n'existent pas (`CREATE TABLE IF NOT EXISTS`).
- **Crée un compte admin** de départ (« seed ») à partir du `.env`.

```
import Database from 'better-sqlite3';
const db = new Database(path.resolve(dbPath)); // ouvre le fichier

db.exec(`
  CREATE TABLE IF NOT EXISTS taches (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    titre       TEXT NOT NULL,
    description  TEXT,
    statut      TEXT DEFAULT 'todo',
    assignee    TEXT,
    created_at  TEXT DEFAULT (datetime('now'))
  );
  /* ... 10 autres tables ... */
`);
export default db;
```

Pourquoi better-sqlite3 ? Son API est **synchrone** : on écrit `db.prepare(...).get()` et on a le résultat tout de suite, sans `await`. Beaucoup plus simple à lire pour débiter, et assez rapide pour une application de cette taille.

Les concepts SQL à retenir

- **PRIMARY KEY AUTOINCREMENT** : chaque ligne reçoit un `id` unique automatique.
- **NOT NULL** : ce champ est obligatoire (ex : `titre`). La base refuse une valeur vide.
- **DEFAULT 'todo'** : si on ne précise rien, la valeur par défaut est utilisée.
- **parent_id** (table `dossiers`) : une table qui pointe vers elle-même, pour faire une arborescence de dossiers.

Les tables de Lexora

Table	Contenu	Champs clés
<code>taches</code>	Tâches projet	titre, statut, assignee
<code>todos</code>	Notes rapides	titre, priorite, statut
<code>factures</code>	Factures clients	client, montant, statut, dates
<code>clients</code>	Particuliers et entreprises	type_client, email, nom / raison_sociale
<code>employes</code>	Équipe + comptes de connexion	email, role, password_hash
<code>planning</code>	Créneaux horaires	employe, date, heure_debut/fin
<code>evenements</code>	Calendrier	titre, date_debut, type, couleur
<code>automations</code>	Règles d'automatisation	nom, action, actif (1/0)
<code>dossiers</code>	Arborescence du coffre-fort	nom, parent_id
<code>documents</code>	Métadonnées des fichiers	nom, nom_fichier, dossier_id

6. L'anatomie d'une route (le cœur du métier)

Toutes les routes se ressemblent. Si tu comprends `taches.js`, tu les comprends toutes. Étudions ses 4 opérations **CRUD** (Create, Read, Update, Delete).

READ — Lire toutes les tâches

```
router.get('/', (req, res) => {
  try {
    const taches = db.prepare('SELECT * FROM taches ORDER BY created_at DESC').all();
    res.json(taches); // renvoie le tableau en JSON
  } catch (err) {
    res.status(500).json({ error: 'Erreur serveur' });
  }
});
```

- `router.get('/')` = répond à `GET /api/taches` (le préfixe vient d'`index.js`).
- `req` = la **requête** reçue ; `res` = la **réponse** qu'on renvoie.
- `.all()` récupère **toutes** les lignes ; `.get()` en récupère **une seule**.
- `try/catch` : si la base plante, on renvoie une erreur **500** au lieu de faire crasher le serveur.

CREATE — Ajouter une tâche

```
router.post('/', (req, res) => {
  try {
    const { titre, description, statut, assignee } = req.body; // 1. lire les données
    if (!titre) return res.status(400).json({ error: 'titre requis' }); // 2. validation

    const result = db.prepare(
      'INSERT INTO taches (titre, description, statut, assignee) VALUES (?, ?, ?, ?)'
    ).run(titre, description || null, statut || 'todo', assignee || null); // 3. insertion

    const tache = db.prepare('SELECT * FROM taches WHERE id = ?').get(result.lastInsertRowId);
    res.status(201).json(tache); // 4. renvoie la tâche créée (201 = Créé)
  } catch (err) {
    res.status(500).json({ error: 'Erreur serveur' });
  }
});
```

Les ? dans le SQL = requêtes préparées. On ne colle JAMAIS les données utilisateur directement dans le texte SQL. Les `?` sont remplacés en toute sécurité par `.run(...)`. C'est ce qui protège contre les *injections SQL* (un pirate qui glisse du code malveillant dans un champ texte).

Les **étapes 1-2-3-4** (lire -> valider -> écrire -> répondre) se répètent dans presque toutes les routes POST/PUT du projet.

UPDATE — Modifier une tâche

```
router.put('/:id', (req, res) => {
  const { titre, description, statut, assignee } = req.body;
  if (!titre) return res.status(400).json({ error: 'titre requis' }); // valider AVANT

  const result = db.prepare(
    'UPDATE taches SET titre=?, description=?, statut=?, assignee=? WHERE id=?'
  ).run(titre, description, statut, assignee, req.params.id);
```

```

if (result.changes === 0) // aucune ligne modifiée = id inexistant
  return res.status(404).json({ error: 'Tâche non trouvée' });
...
});

```

- `:id` est un **paramètre d'URL**. Dans `PUT /api/taches/5`, `req.params.id` vaut `"5"`.
- `result.changes` indique combien de lignes ont été touchées. `0` -> l'id n'existe pas -> erreur **404**.
- On valide le titre **avant** l'UPDATE : sinon on risquerait d'écrire un titre vide alors que la colonne est **NOT NULL** (bug corrigé dans le code).

DELETE — Supprimer une tâche

```

router.delete('/:id', (req, res) => {
  const result = db.prepare('DELETE FROM taches WHERE id = ?').run(req.params.id);
  if (result.changes === 0) return res.status(404).json({ error: 'Tâche non trouvée' });
  res.json({ success: true });
});

```

Mémo des codes HTTP : `200` OK · `201` Créé · `400` Données invalides · `401` Pas connecté · `404` Introuvable · `409` Conflit (ex : email déjà pris) · `500` Erreur serveur.

7. Les routes un peu spéciales

7.1 — La connexion : auth.js

```
router.post('/login', (req, res) => {
  const { email, password } = req.body;
  const employee = db.prepare('SELECT * FROM employes WHERE email = ?').get(email);
  if (!employee || !employee.password_hash)
    return res.status(401).json({ error: 'Identifiants incorrects' });

  const valid = bcrypt.compareSync(password, employee.password_hash); // compare le hash
  if (!valid) return res.status(401).json({ error: 'Identifiants incorrects' });

  const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '24h' });
  res.json({ token, user: payload }); // renvoie le badge + infos user
});
```

`bcrypt.compareSync` rehash le mot de passe saisi et le compare au hash stocké : on ne déchiffre jamais le mot de passe d'origine. Si tout est bon, `jwt.sign` fabrique le token.

7.2 — L'upload de fichiers : documents.js

C'est la route la plus riche. Elle utilise **Multer** pour recevoir un vrai fichier.

```
router.post('/upload', upload.single('file'), (req, res) => {
  if (!req.file) return res.status(400).json({ error: 'Aucun fichier reçu' });
  ...
});
```

- `upload.single('file')` est un middleware : Multer enregistre le fichier sur le disque (dossier `uploads/`) AVANT que ta fonction s'exécute.
- On ne stocke **pas** le fichier dans la base : seulement ses **métadonnées** (nom, taille, type). Le fichier physique reste sur le disque.
- La suppression d'un dossier est **récursive** (`supprimerDossierRecuratif`) : elle efface aussi tous les sous-dossiers et leurs fichiers.

7.3 — L'assistant IA : assistant.js + ollamaService.js

Ici le backend joue un rôle de « **proxy** » (intermédiaire) : il transmet la question à un modèle d'IA local (Ollama) et renvoie la réponse.

```
Frontend -> POST /api/assistant { prompt } -> backend
backend -> chat(prompt) -> Ollama (port 11434) -> réponse texte
backend -> renvoie { response } -> Frontend
```

Ce découpage est important : le frontend ne parle jamais directement à Ollama. Le backend sert d'intermédiaire, ce qui permet de gérer les erreurs proprement (si Ollama est éteint -> erreur 500 avec un message clair).

8. Le lien Frontend ↔ Backend (le plus important !)

Voici la question centrale : **comment un clic dans React déclenche-t-il du code backend ?**

Suivons un exemple réel de bout en bout : *ajouter une tâche*.

Étape 1 — Le frontend appelle l'API (frontend/src/pages/Taches.jsx)

```
const handleSubmit = async e => {
  e.preventDefault(); // empêche le rechargement de page
  await fetch('/api/taches', { // <- appel HTTP vers le backend
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(form), // les données du formulaire en JSON
  });
  await load(); // recharge la liste
};
```

`fetch()` est la fonction du navigateur qui envoie une requête HTTP. Ici : verbe `POST`, URL `/api/taches`, corps = le formulaire transformé en JSON.

Étape 2 — Mais... /api/taches sans adresse complète ?

Le frontend tourne sur le port 5173, le backend sur le 3000. Comment l'appel arrive-t-il au bon endroit ? Grâce au **proxy de Vite** (frontend/vite.config.js) :

```
server: {
  proxy: {
    '/api': {
      target: 'http://localhost:3000', // tout ce qui commence par /api
      changeOrigin: true, // est redirigé vers le backend
    },
  },
}
```

C'est pour ça qu'on écrit juste `/api/taches` dans React (et pas `http://localhost:3000/api/taches`) : Vite redirige automatiquement. En production, c'est Nginx (nginx.conf) qui joue ce rôle.

Étape 3 — Le voyage complet de la requête

```
Taches.jsx      fetch('/api/taches', POST)
  v proxy Vite redirige vers localhost:3000
index.js        helmet -> cors -> rate-limit -> express.json()
  v app.use('/api/taches', tachesRouter)
routes/taches.js router.post('/') s'exécute
  v INSERT INTO taches ...
lexora.db       la ligne est enregistrée
  ^ res.status(201).json(tache)
Taches.jsx      reçoit la réponse, recharge la liste, affiche un toast
```

Étape 4 — Cas d'une route protégée (avec token)

Pour les routes `/api/employes`, le frontend doit joindre son badge. Le `AuthContext.jsx` fournit un raccourci `authHeaders` :

```
// AuthContext.jsx
const authHeaders = token ? { Authorization: `Bearer ${token}` } : {};

// dans une page qui liste les employés
fetch('/api/employes', { headers: authHeaders })
```

Côté backend, le middleware `verifyJWT` lit cet en-tête `Authorization`, vérifie le token, et autorise (ou refuse) l'accès. La boucle est bouclée.

Récapitulatif : quelle page appelle quelle route ?

Page Frontend	Route Backend appelée	Protégée ?
Login.jsx	POST /api/auth/login	—
Taches.jsx	/api/taches, /api/todos	Non
ClientsFactures.jsx	/api/clients, /api/factures	Non
Calendrier.jsx	/api/evenements, /api/planning	Non
Coffre_fort.jsx	/api/documents (+ upload)	Non
Equipe.jsx	/api/employes	Oui (JWT)
Assistant.jsx	POST /api/assistant	Non

9. Pour lancer le projet et expérimenter

Démarrer le backend

```
cd lexora/backend
npm install          # installe les dépendances (une seule fois)
cp ../.env.example ../.env  # crée ton fichier de config
npm run dev          # démarre avec rechargement auto (--watch)
```

Tester une route sans frontend (avec curl)

```
# Lire toutes les tâches
curl http://localhost:3000/api/taches

# Créer une tâche
curl -X POST http://localhost:3000/api/taches \
  -H "Content-Type: application/json" \
  -d '{"titre":"Ma première tâche"}'

# Vérifier que le serveur est vivant
curl http://localhost:3000/api/health
```

Ton premier exercice conseillé : ajoute une colonne **priorite** à la table **taches** dans db.js, puis modifie routes/taches.js pour la lire/écrire. Tu auras touché toute la chaîne backend en une fois.

10. Le vocabulaire à retenir

Terme	Définition courte
API REST	Ensemble d'URL appelables avec des verbes HTTP.
Route / endpoint	Une URL + un verbe associés à du code (ex : POST /api/taches).
Middleware	Fonction exécutée avant la route (sécurité, parsing, auth...).
CRUD	Create, Read, Update, Delete : les 4 opérations de base.
JSON	Format texte d'échange de données.
JWT / token	« Badge » signé prouvant qu'on est connecté.
Hash (bcrypt)	Transformation irréversible d'un mot de passe.
Requête préparée	SQL avec des ? pour éviter les injections.
Proxy	Intermédiaire qui redirige les requêtes (Vite, ou backend -> Ollama).
req / res	Objets « requête reçue » et « réponse à envoyer » dans Express.

— Fin du guide. Bon courage pour tes débuts en backend ! —